Eleição em Anel e Two-Phase Commit para Transferências Bancárias Distribuídas

Integrante 1: *[Nome Completo]*
RA: *[Número de RA]*
Instituto de Computação – Unicamp
MC714 – Sistemas Distribuídos (2ºs/2026)

Integrante 2: *[Nome Completo]*
RA: *[Número de RA]*
Instituto de Computação – Unicamp
MC714 – Sistemas Distribuídos (2ºs/2026)

Entregáveis do Projeto
GitLab: https://gitlab.ic.unicamp.br/mc714/2026s2/raXXXXXX
Vídeo: https://youtu.be/exemplo-video-mc714
Demonstração em vídeo: $\approx$ 5 minutos

***Abstract*—Este trabalho apresenta a implementação e a avaliação experimental de dois algoritmos distribuídos clássicos e complementares: a eleição de líder em anel e o protocolo de compromisso atômico *Two-Phase Commit* (2PC), integrados em uma aplicação de transferências bancárias replicadas. O sistema executa sobre contêineres OCI independentes orquestrados por Docker Compose v2, configurados em topologias de 3, 5 e 10 nós lógicos, comunicando-se exclusivamente por gRPC sobre TCP com prazos de expiração (*deadlines*) explícitos. O líder eleito pelo anel assume o papel de coordenador do 2PC, enquanto os demais nós operam como coortes participantes com controle estrito de concorrência e travas (*locks*) de contas em memória. A suíte automatizada de testes validou propriedades de segurança e vivacidade em fluxos normais de commit, abort por saldo insuficiente, tolerância a crash de participantes na Fase 1 e a ocorrência do clássico problema do bloqueio (*blocking problem*) sob queda do coordenador pós-*prepare*, demonstrando sua resolução operacional autônoma através da reeleição no anel. Os ensaios quantitativos evidenciaram latências médias de 2PC de 6,73 ms, 10,46 ms e 20,01 ms para 3, 5 e 10 nós, respectivamente, com detalhamento de percentis (p50, p95 e p99) e discussão da complexidade de mensagens.**

***Index Terms*—sistemas distribuídos, eleição em anel, two-phase commit, gRPC, detecção de falhas, tolerância a falhas.**

I. Introdução e Motivação

A coordenação de processos autônomos e a garantia de atomicidade em operações de escrita constituem dois dos desafios mais fundamentais no projeto de sistemas distribuídos modernos [1]. Em ambientes replicados onde múltiplas entidades compartilham a responsabilidade de manter a coerência de um estado global, a ausência de um coordenador central fixo exige protocolos robustos para a eleição dinâmica de líderes. Paralelamente, quando uma transação envolve mutações indivisíveis que abrangem múltiplos nós, protocolos de compromisso atômico tornam-se indispensáveis para assegurar que todos os nós convirjam para o mesmo desfecho (todos aplicam a alteração ou todos abortam), eliminando estados parciais inconsistentes.

Neste projeto, implementamos e investigamos empiricamente o Algoritmo de Eleição de Líder em Anel e o protocolo *Two-Phase Commit* (2PC), integrando-os de maneira deliberada em uma aplicação prática de transferências financeiras entre réplicas contábeis. A escolha desse domínio bancário reduz a complexidade acessória de regras de negócio sem descaracterizar a propriedade central de interesse: a transferência entre contas (`conta_A` e `conta_B`) deve preservar estritamente a conservação do valor monetário total e a consistência mútua entre todas as réplicas ativas, garantindo isolamento através de travas (*locks*) de conta.

A integração entre ambos os algoritmos resolve uma fragilidade inerente ao 2PC puro. No 2PC tradicional, o coordenador representa um ponto único de falha (*single point of failure*); caso o coordenador colapse após os participantes votarem SIM na Fase 1 (*prepare*) e antes de emitir a decisão da Fase 2, os participantes entram em estado de incerteza e bloqueio permanente, incapazes de decidir unilateralmente sem violar a segurança. Ao acoplar um detector de falhas baseado em *heartbeats* periódicos e o algoritmo de eleição em anel, nosso sistema detecta a inatividade do coordenador, elege dinamicamente um novo líder e executa uma rotina cooperativa de resolução da pendência, restaurando a vivacidade do cluster.

O artefato foi desenvolvido em Python 3.10 com comunicação estritamente baseada em stubs gerados por `grpcio` v1.59.0 a partir de contratos Protocol Buffers (`proto3`), sendo orquestrado por contêineres Docker independentes. Conforme as diretrizes da disciplina, nenhum componente de terceiros que implemente os algoritmos distribuídos foi utilizado; toda a lógica do anel, coordenação do 2PC, transações contábeis e mecanismos de injeção de caos foi concebida e implementada integralmente pelos autores.

II. Modelo do Sistema e Propriedades de Correção

A especificação formal do modelo sob o qual os algoritmos operam é pré-requisito mandatório para delimitar garantias teóricas e interpretar resultados empíricos. A seguir, declaramos e justificamos os pressupostos para cada um dos dois algoritmos desenvolvidos.

A. Algoritmo de Eleição de Líder em Anel

1. Modelo Temporal: O modelo adotado é *parcialmente síncrono*. Não se assume a existência de cotas superiores rígidas a priori para o tempo de transmissão em rede local; contudo, após estabilização, os prazos de expiração (*deadlines* de RPC de 2,0 s) e os intervalos de *heartbeat* (3,0 s com timeout de 1,5 s) fornecem limites práticos confiáveis para suspeitar da inatividade de vizinhos ou do líder, evitando bloqueios indefinidos.

2. Propriedades dos Canais: A comunicação utiliza gRPC sobre conexões TCP ponto a ponto, garantindo transporte ordenado (FIFO) e detecção de perdas por canal. Não há ordenação causal global entre conexões distintas. Chamadas expiradas retornam `DEADLINE_EXCEEDED` ou `UNAVAILABLE`, sendo interpretadas como suspeita de inoperância.

3. Tipos de Falha Considerados: Considera-se falhas *crash-stop* e *crash-recovery*. O anel tolera a queda de nós intermediários através de salto dinâmico (*skip*): ao detectar um sucessor imediato inativo, avança-se na topologia circular para o próximo nó disponível. Quedas do líder são diagnosticadas após 2 falhas consecutivas de heartbeat. Não são admitidas falhas bizantinas nem partições de rede desconexas.

4. Hipóteses sobre Relógios, Persistência e Reinicialização: Não se presume sincronização de relógios físicos; utilizam-se apenas timers monotônicos locais para timeouts. Identificadores pseudoaleatórios não sequenciais são gerados no boot. O anel opera em memória volátil, recalculando a topologia ativa a cada ciclo eleitoral.

5. Propriedades de Segurança e Vivacidade:

- *Segurança (Unicidade e Concordância):* Ao término de uma eleição, todos os nós alcançáveis concordam sobre o mesmo líder, rigorosamente o de maior identificador ativo: $\forall p, q \in \text{Vivos},\ \text{Líder}_p = \text{Líder}_q = \max(\{ID_i\})$.
- *Vivacidade (Término):* Qualquer eleição deflagrada em topologia com ao menos um nó ativo e caminhos alcançáveis eventualmente encerra com o anúncio do coordenador a todos os nós.

B. Protocolo Two-Phase Commit (2PC) Bancário

1. Modelo Temporal: Parcialmente síncrono. O coordenador adota deadlines gRPC de 2,5 s na Fase 1 (*prepare*) e Fase 2. Qualquer expiração na Fase 1 é tratada conservadoramente como voto *NÃO*, priorizando a segurança sobre a disponibilidade.

2. Propriedades dos Canais: RPCs ponto a ponto sobre gRPC/TCP. Os participantes mantêm histórico de transações decididas (`transaction_history`), garantindo *idempotência* frente a retentativas de rede.

3. Tipos de Falha Considerados: Tolera-se crash-stop de coortes na Fase 1 (resultando em abort seguro). Queda do coordenador pós-*prepare* (o clássico *blocking problem*) retém locks em `PREPARED` até resolução pelo novo líder. Nós recuperados sincronizam saldos via `SyncBankState`.

4. Hipóteses sobre Relógios, Persistência e Reinicialização: Sem relógios sincronizados. Travas locais (`bank_lock`) gerenciam concorrência. Saldos confirmados e histórico são persistidos em snapshot JSON (`/tmp/bank_state_*.json`) na Fase 2, permitindo recuperação de estado após reinicialização.

5. Propriedades de Segurança e Vivacidade:

- *Segurança (Atomicidade e Consistência):* (i) Se uma réplica comita, todas as operacionais comitam; caso contrário, todas abortam; (ii) Nenhuma transação comita sem voto SIM unânime; (iii) A soma dos saldos de todas as contas é invariante ($\sum_c \text{Saldo}(c) = \text{constante}$); (iv) Locks exclusivos impedem disputas concorrentes sobre contas em `PREPARED`.
- *Vivacidade (Progresso):* Transações sem falha de coordenador concluem em `COMMITTED` ou `ABORTED`. Sob crash pós-*prepare*, o bloqueio é destravado assim que o anel elege um novo líder para conduzir a resolução.

III. Arquitetura e Algoritmos

Cada nó executa em contêiner independente (`src/node.py`) instanciando um servidor gRPC multithread (`ThreadPoolExecutor`) com os serviços `RingService` e `TwoPhaseCommitService`.

A. Topologia e Descoberta de Vizinhos

A ordem nominal do anel é passada por `RING_TOPOLOGY` (ex: `no_1:50051, ...`). O nó determina sua lista circular de sucessores a partir de `MY_ADDR`. Cada contêiner sorteia um identificador pseudoaleatório de 32 bits no boot, tornando a eleição não-trivial e desacoplada dos nomes dos nós.

B. Algoritmo do Anel com Salto Dinâmico

A eleição opera segundo a variante de van Steen e Tanenbaum [1] com tolerância a falhas:

1) **Deflagração:** O iniciador cria `ElectionMessage` com seu identificador e a transmite via `PassElection` ao primeiro sucessor responsivo.
2) **Salto Dinâmico (*Skip*):** Se o sucessor imediato não responder dentro do timeout (2,0 s), o nó remetente itera sobre os sucessores seguintes, saltando contêineres inativos até alcançar um nó vivo.
3) **Fechamento de Ciclo:** Cada intermediário adiciona seu par `(id, addr)` e repassa a mensagem. Ao retornar ao iniciador, este elege o maior identificador do circuito.
4) **Anúncio:** O iniciador difunde `AnnounceCoordinator` pelo anel. Cada participante atualiza seu líder e repassa a mensagem.

C. Detecção de Falhas via Heartbeat Periódico

Em segundo plano, uma thread em cada nó seguidor emite a RPC `Ping` ao líder a cada 3,0 s (timeout 1,5 s). Se o líder falhar 2 vezes consecutivas (`MISSED_HEARTBEATS_LIMIT`), o nó assume a queda do líder e dispara nova eleição no anel.

D. Protocolo Two-Phase Commit Bancário

O líder atua como Coordenador do 2PC e os demais nós como coortes:

- **Submissão:** O cliente invoca `SubmitTransaction` no líder (nós seguidores efetuam proxy transparente ao líder).

- **Fase 1 (Prepare):** O coordenador despacha paralelamente `PrepareTransaction`. As coortes validam contas, valor positivo, ausência de locks ativos e saldo suficiente. Em caso de aprovação, retêm os locks das contas em `pending_transactions` e votam SIM; havendo falha ou timeout, votam NÃO.
- **Fase 2 (Commit/Abort):** Com votos SIM unânimes, o coordenador difunde `CommitTransaction`, efetivando débitos/créditos, liberando locks e gravando o snapshot local. Se houver voto NÃO ou timeout, difunde `AbortTransaction`, descartando o preparo e liberando as contas sem alterar saldos.

E. Resolução Autônoma do Bloqueio Pós-Crash do Líder

A injeção de crash no líder entre as fases retém coortes no estado `PREPARED`. O detector de heartbeat constata a queda e o anel elege um novo líder. O novo coordenador invoca `ResolveBlockedTransaction` com `ABORT`, liberando os locks de forma segura. Ao ser reiniciado, o antigo líder executa `SyncBankState` para sincronizar saldos com as réplicas ativas e purgar locks órfãos.

IV. Validação de Correção e Testes

A correção foi comprovada por testes automatizados reproduzíveis cobrindo cenários normais e de falha para ambos os algoritmos.

A. Correção da Eleição em Anel

- **Cenário Normal (Eleição em Topologia Íntegra):** Com 10 nós ativos, uma eleição foi disparada a partir do `no_1`. A mensagem completou 10 saltos e o nó com maior identificador foi eleito e anunciado a 100% dos nós, conforme verificado via `GetStatus`.
- **Cenário de Falha de Nó Intermediário (*Skip*):** O contêiner `no_3` foi parado (`docker stop no_3`). Ao deflagrar eleição via `no_2`, o nó detectou a indisponibilidade no timeout e saltou para `no_4`, elegendo o líder correto entre os 9 nós vivos.
- **Cenário de Falha do Coordenador via Heartbeat:** O líder ativo foi encerrado. Após 6,0 s (2 heartbeats perdidos), os seguidores diagnosticaram a queda e elegeram autonomamente um novo líder.

B. Correção do Two-Phase Commit Bancário

A suíte `src/test_2pc_scenarios.py` foi executada sobre 10 nós em 11/11/2026, com 100% de êxito em 18,60 s:

1) **Commit Normal:** Transferência válida de R$100,00 de `conta_A` para `conta_B`; todas as 10 réplicas confirmaram saldos de R$800,00 e R$1200,00, sem travas residuais e com soma total invariante.
2) **Abort por Regra de Negócio:** Tentativa de transferência de R$51.000,00 (saldo insuficiente); coortes emitiram voto NÃO na Fase 1, resultando em abort global com saldos e locks preservados.
3) **Crash de Participante na Fase 1:** Coorte `no_4` pausada via `SetChaosHook`; timeout no coordenador tratado como voto NÃO, cancelando a operação sem alterar saldos dos sobreviventes.
4) **Crash do Coordenador pós-Prepare (Bloqueio):** Queda induzida no líder pós-coleta de votos SIM; coortes comprovaram o bloqueio teórico em `PREPARED`. O anel elegeu novo líder, que acionou `ResolveBlockedTransaction`, liberando os locks com sucesso. O nó caído reiniciou e sincronizou saldos via `SyncBankState`.

TABLE I
CONVERGÊNCIA DA ELEIÇÃO EM ANEL (MS, 20 RODADAS).

Nós	Sucesso	Média	DP	p50	p95	p99
3	20/20	107,57	2,05	107,00	110,77	111,45
5	20/20	107,65	1,35	107,31	109,08	109,80
10	20/20	107,56	1,23	107,23	108,50	109,20

TABLE II
LATÊNCIA PONTA A PONTA DO 2PC (MS, 20 COMMITS).

Nós	Sucesso	Média	DP	p50	p95	p99
3	20/20	6,73	0,78	6,52	7,68	8,99
5	20/20	10,46	1,21	10,17	12,83	14,02
10	20/20	20,01	2,16	19,31	24,18	26,16

V. Metodologia Experimental e Ambiente

Avaliou-se a escalabilidade dos dois algoritmos manipulando o tamanho do cluster (N) em três níveis: **3, 5 e 10 nós lógicos** (`docker/compose-3nodes.yaml`, `compose-5nodes.yaml` e `compose.yaml`).

As métricas analisadas foram: (i) **Tempo de Convergência da Eleição (ms)**, do disparo da eleição até o encerramento do anúncio; e (ii) **Latência do 2PC (ms)**, do envio do *prepare* até a confirmação da Fase 2. Executaram-se 20 rodadas independentes ($R = 20$) por topologia, com alternância de contas e valores (R$1 a R$10) e intervalo de 0,3 s entre transações.

Ambiente de Execução: Processador AMD Ryzen 5 5600 6-Core (4 vCPUs alocadas, 3,8 GiB RAM DDR4), Debian Linux (Kernel 6.12.116), Docker Engine 29.9.3, Compose v2, imagem `python:3.10-slim`, `grpcio 1.59.0`. Duração agregada dos ensaios de $\approx$ 110 s sem aquecimento prévio separado. Estatísticas (Média, DP, p50, p95 e p99) obtidas por interpolação linear.

VI. Resultados e Discussão

Os dados experimentais consolidados constam nas Tabelas I e II.

A. Análise do Protocolo 2PC Bancário

A Tabela II evidencia um aumento estrito da latência média com o número de coortes: 6,73 ms (3 nós), 10,46 ms (5 nós) e 20,01 ms (10 nós), um salto de quase $3\times$ entre 3 e 10 nós. Para N nós, o 2PC requer $2(N-1)$ mensagens entre coordenador e coortes ($O(N)$ mensagens). Como a Fase 1 e

a Fase 2 aguardam a última resposta no caminho crítico, o acréscimo de participantes eleva a probabilidade de atrasos por chaveamento de threads e contenção de CPU, expandindo a cauda de latência: o p99 aumentou de 8,99 ms para 26,16 ms. A taxa de êxito foi de 100% nas 60 transações.

B. Análise da Eleição em Anel e Limitações de Medição

Na Tabela I, as médias permaneceram estabilizadas em $\approx 107,6$ ms em todas as escalas. Embora a teoria dite $2N$ encaminhamentos ($O(N)$ saltos), o script cliente consulta o status por amostragem periódica de 100 ms (`time.sleep(0.1)`). Como a propagação das mensagens gRPC sobre a rede loopback local consome menos de 10 ms mesmo em 10 nós, o valor registrado reflete a granularidade do *polling* somada ao handshake gRPC, mascarando a complexidade assintótica do algoritmo e demonstrando a relevância de registrar carimbos temporais internos aos nós.

VII. Ameaças à Validade e Limitações

- **Contêineres em Host Único:** Os nós compartilham a mesma CPU física, memória e kernel Linux, isolando processos mas não constituindo domínios de falha independentes.
- **Persistência e Semântica de Recuperação:** O snapshot de saldos em JSON permite recuperação rápida de saldos, mas transações em trânsito e locks residem em memória volátil, sem Write-Ahead Log com `fsync` síncrono.
- **Partições Assimétricas:** Quedas prolongadas que fragmentem o anel paralisam a circulação de mensagens na ausência de mecanismos de quórum.

VIII. Itens Extras e Atividades de Bônus

Conforme a Seção 5 do regulamento, o projeto implementou três extensões avançadas: **1) Integração Coerente em Aplicação Prática:** O líder do anel coordena o 2PC e a reeleição é usada ativamente para destravar transações bloqueadas pós-queda; **2) Terceiro Protocolo (Detecção de Falhas por Heartbeat):** Detector autônomo baseado em RPCs `Ping` periódicas com contadores de perda configuráveis; **3) Mecanismo de Crash-Recovery e Reconciliação:** Implementação de `SyncBankState` para restauração de saldos e eliminação de travas órfãs em nós reiniciados.

IX. Declaração de Autoria e Código Reutilizado

Toda a arquitetura, lógica dos algoritmos de anel com salto dinâmico (*skip*), protocolo 2PC, transações contábeis, heartbeat, testes e scripts experimentais foram concebidos e implementados integralmente pelos membros da equipe. Utilizou-se unicamente bibliotecas padrão do Python 3.10 e `grpcio`/`grpcio-tools` v1.59.0 para transporte RPC e geração de stubs dos arquivos `.proto`. Nenhuma biblioteca de consenso ou commit atômico de terceiros foi empregada.

X. Instruções de Reprodução

Todo o ambiente é reprodutível via Docker Compose v2: inicialização com `docker compose up -d --build`; validação com `docker compose exec no_1 python src/test_2pc_scenarios.py`; coleta experimental com `src/run_experiments.py` e `src/run_2pc_experiments.py` (`--rounds 20`); topologias com `-f docker/compose-3nodes.yaml` ou `5nodes.yaml`; e encerramento com `docker compose down -v`.

XI. Conclusão

A integração entre eleição em anel e 2PC permitiu materializar o compromisso clássico entre disponibilidade e atomicidade estrita em sistemas distribuídos. O detector de falhas e a reeleição proporcionam uma recuperação operacional prática para o problema do bloqueio, enquanto as medições quantitativas evidenciaram o impacto da escala sobre a cauda de latência do 2PC e a necessidade de instrumentação de alta resolução em avaliações de sistemas concorrentes.

References

[1] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed., distributed-systems.net, 2023.

[2] gRPC Authors, "gRPC Python Documentation," 2026. [Online]. Disponível em: https://grpc.io/docs/languages/python/. Acesso em: 11-Nov-2026.

[3] Docker Inc., "Docker Compose Documentation v2," 2026. [Online]. Disponível em: https://docs.docker.com/compose/. Acesso em: 11-Nov-2026.